

Report - Design Rationale

251163014

Project: Bhasha Quest - Language Learning Application for Indic Scripts

CS3307A Deliverable 2

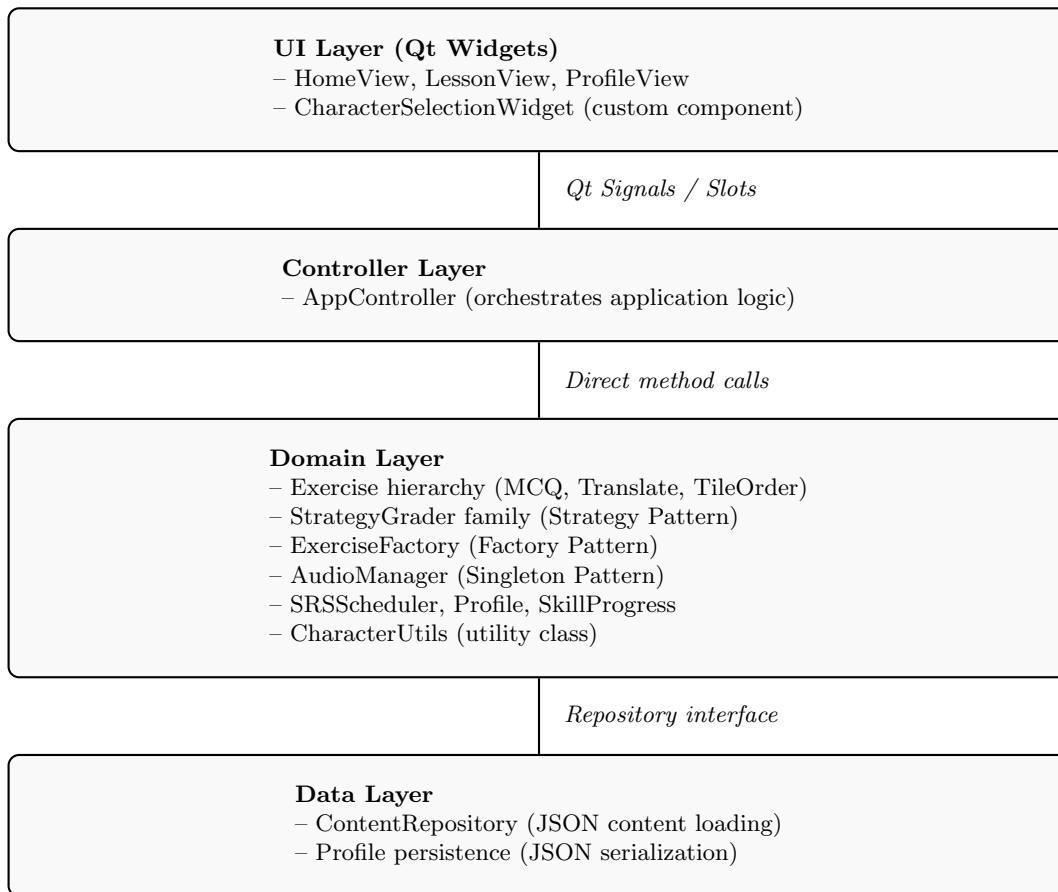
Contents

1	Architecture Overview	2
1.1	System Structure	2
1.2	Architectural Principles	2
1.3	Key Architectural Benefits	3
2	Design Patterns	4
2.1	Factory Pattern: <code>ExerciseFactory</code>	4
2.2	Strategy Pattern: <code>StrategyGrader</code> Hierarchy	5
2.3	Singleton Pattern: <code>AudioManager</code>	6
3	Updated Architecture & Flow Diagrams (Deliverable 2)	7
3.1	Class Diagram: Layers, Responsibilities, and Patterns	7
3.2	Sequence Diagram: “Complete Exercise” Orchestration	8
4	Implementation Challenges	10
4.1	Challenge 1: Qt 6 Installation and Learning Curve	10
4.2	Challenge 2: qmake vs. CMake Build System Confusion	10
4.3	Challenge 3: Indic Script Input Solution – <code>CharacterSelectionWidget</code> Innovation	10
4.4	Challenge 4: Qt Signal/Slot Wiring Complexity	11
5	Changes from Deliverable 1	11
5.1	Addition 1: <code>CharacterSelectionWidget</code> (Beyond Requirements)	11
5.2	Addition 2: <code>CharacterUtils</code> Helper Class	12
5.3	Addition 3: Enhanced UI with Animations and Modern Styling	12
5.4	Addition 4: Time Travel Dev Tools for Testing	13
6	Next Steps for Deliverable 3	14
6.1	Feature 1: <code>ListenExercise</code> with Audio Playback	14
6.2	Feature 2: <code>SpeakExercise</code> with Recording and Stub ASR	14
6.3	Feature 3: Observer Pattern for Real-Time Updates	14
6.4	Feature 4: GoogleTest / GMock Unit Tests	15
6.5	Feature 5: SQLite Persistence Option	15
7	Reflection	16

1 Architecture Overview

1.1 System Structure

Bhasha Quest employs a layered MVC (Model-View-Controller) architecture using Qt 6 and C++17, with strict adherence to the separation of concerns principle. The application is organized into four distinct architectural layers:



1.2 Architectural Principles

Strict Layer Separation

The architecture enforces a unidirectional dependency flow:

$$\text{UI} \rightarrow \text{Controller} \rightarrow \text{Domain} \rightarrow \text{Data}$$

The domain layer has zero knowledge of the UI implementation, ensuring that business logic remains testable and reusable. UI components communicate exclusively with **AppController** through Qt's signal/slot mechanism, never directly accessing domain objects.

Qt Signal/Slot Mechanism for Loose Coupling

The UI layer connects to **AppController** using Qt's signal/slot system, enabling loose coupling between presentation and control logic. When a user submits an answer, **LessonView** emits a signal that **AppController** receives through a slot, processes via domain logic, and emits result signals back to the UI. This decoupling allows UI modifications without touching controller code, and vice versa.

Controller as Orchestrator

`AppController` serves as the central orchestrator, managing lesson flow, exercise progression, and user profile updates. It delegates business logic to domain classes (e.g., `StrategyGrader` for evaluation, `SRSScheduler` for review intervals) while maintaining session state. This separation ensures that complex workflows remain comprehensible and domain objects stay focused on single responsibilities.

Repository Pattern for Data Abstraction

`ContentRepository` provides a clean abstraction over data storage, currently implemented with JSON file loading. The repository interface shields the domain layer from data source details, allowing future migration to SQLite or cloud storage without modifying domain classes. Profile serialization follows the same principle, using JSON as the persistence mechanism with clear boundaries between business objects and their serialized representations.

1.3 Key Architectural Benefits

1. **Testability:** Domain logic can be unit-tested in isolation without Qt dependencies.
2. **Maintainability:** Changes to the UI design do not ripple into business logic, ensuring stability across layers.
3. **Extensibility:** New exercise types can be added by extending the `Exercise` base class and creating corresponding grader implementations.
4. **Scalability:** Well-defined layer boundaries enable parallel development across independent components, improving team efficiency.

2 Design Patterns

2.1 Factory Pattern: ExerciseFactory

Pattern Type: Creational

Implementation Location: src/core/domain/ExerciseFactory.h/cpp

Why This Pattern Was Chosen

The application supports multiple exercise types (`MCQExercise`, `TranslateExercise`, `TileOrderExercise`) that share a common interface but have different creation requirements. Without the Factory Pattern, the `ContentRepository` would need complex conditional logic to instantiate exercises based on type strings from JSON data. This would violate the Open/Closed Principle and create tight coupling between data loading and exercise instantiation.

How It Was Implemented

```
1 class ExerciseFactory {
2 public:
3     static Exercise* createExercise(const QString& type, const QJsonObject& spec);
4
5 private:
6     static Exercise* createMCQ(const QJsonObject& spec);
7     static Exercise* createTranslate(const QJsonObject& spec);
8     static Exercise* createTileOrder(const QJsonObject& spec);
9 };
```

Listing 1: ExerciseFactory Implementation

The factory provides a single static `createExercise()` method that accepts an exercise type string ("MCQ", "Translate", "TileOrder") and a JSON specification. It delegates to private helper methods for type-specific construction, encapsulating the details of how each exercise type is initialized from JSON data.

Usage Example

```
1 // In ContentRepository::loadExercises()
2 QString exerciseType = jsonObj["type"].toString();
3 QJsonObject spec = jsonObj["spec"].toObject();
4 Exercise* exercise = ExerciseFactory::createExercise(exerciseType, spec);
5 exerciseQueue.append(exercise);
```

Listing 2: ExerciseFactory Usage in ContentRepository

Benefits Gained

1. **Encapsulation:** Object creation logic is centralized in one class rather than scattered across the codebase.
2. **Extensibility:** Adding a new exercise type (e.g., `ListenExercise`) requires only modifying `ExerciseFactory`, not `ContentRepository` or `AppController`.
3. **Maintainability:** Changes to exercise construction parameters are localized to factory methods.
4. **Testability:** The factory can be tested independently with mock JSON specifications.

2.2 Strategy Pattern: StrategyGrader Hierarchy

Pattern Type: Behavioral

Implementation Location: `src/core/domain/StrategyGrader.h`, `MCQGrader.cpp`, `TranslateGrader.cpp`, `TileOrderGrader.cpp`, `CharacterSelectionGrader.cpp`

Why This Pattern Was Chosen

Different exercise types require fundamentally different grading algorithms:

- **MCQExercise:** Simple index matching (user-selected option vs. correct index)
- **TranslateExercise:** Fuzzy string matching with multiple acceptable answers
- **TileOrderExercise:** Sequence order verification
- **Character Selection Mode:** Character-by-character sequence matching

Embedding grading logic directly within **Exercise** classes would violate the Single Responsibility Principle and complicate testing. The Strategy Pattern enables grading algorithms to be selected and swapped at runtime based on the exercise type, maintaining clean separation between behavior and data.

How It Was Implemented

```
1 // Abstract strategy interface
2 class StrategyGrader {
3 public:
4     virtual ~StrategyGrader() = default;
5     virtual Result grade(const QString& userAnswer, const Exercise* exercise) = 0;
6 };
7
8 // Concrete strategies
9 class MCQGrader : public StrategyGrader {
10     Result grade(const QString& userAnswer, const Exercise* exercise) override;
11 };
12
13 class TranslateGrader : public StrategyGrader {
14     Result grade(const QString& userAnswer, const Exercise* exercise) override;
15 };
```

Listing 3: StrategyGrader Interface and Concrete Implementations

The AppController dynamically selects the appropriate grader at runtime:

```
1 StrategyGrader* AppController::createGraderForExercise(Exercise* exercise) {
2     if (dynamic_cast<MCQExercise*>(exercise)) {
3         return new MCQGrader();
4     } else if (dynamic_cast<TranslateExercise*>(exercise)) {
5         if (exercise->usesCharacterSelection()) {
6             return new CharacterSelectionGrader();
7         }
8         return new TranslateGrader();
9     } else if (dynamic_cast<TileOrderExercise*>(exercise)) {
10         return new TileOrderGrader();
11     }
12     return nullptr;
13 }
```

Listing 4: Dynamic Strategy Selection in AppController

Benefits Gained

1. **Flexibility:** Grading algorithms can be modified or replaced without altering `Exercise` or `AppController`.
2. **Extensibility:** New grading strategies can be added by implementing the `StrategyGrader` interface.
3. **Testability:** Each grading algorithm can be unit-tested independently with mock `Exercise` objects.
4. **Runtime Selection:** The appropriate strategy is chosen dynamically based on exercise characteristics.

2.3 Singleton Pattern: AudioManager

Pattern Type: Creational

Implementation Location: `src/core/domain/AudioManager.h/cpp`

Why This Pattern Was Chosen

Audio playback relies on resource-intensive Qt components such as `QMediaPlayer` and `QAudioOutput`, which should be initialized once and reused throughout the application's lifecycle. Multiple instances of `AudioManager` would lead to:

- **Resource conflicts:** Multiple players competing for the same audio device.
- **Memory waste:** Duplicate initialization of audio backends.
- **Inconsistent state:** Simultaneous or overlapping playback from multiple players.

The Singleton Pattern ensures a single shared instance manages all audio operations across the system.

How It Was Implemented

```
1 class AudioManager {
2 public:
3     static AudioManager& getInstance();
4
5     // Delete copy/move constructors and assignment operators
6     AudioManager(const AudioManager&) = delete;
7     AudioManager& operator=(const AudioManager&) = delete;
8
9     void playSuccess();
10    void playError();
11    void playAudio(const QString& filename);
12
13 private:
14     AudioManager(); // Private constructor
15     ~AudioManager();
16
17     static AudioManager* instance;
18     QMediaPlayer* player;
19     QAudioOutput* audioOutput;
20 };
```

Listing 5: AudioManager Singleton Implementation

The class exposes a single static access point through `getInstance()`, returning the unique global instance. Its constructor and destructor are private to prevent external instantiation or deletion, and copy/move constructors are deleted to preserve uniqueness.

Usage Example

```

1 // In ApplicationController::submitAnswer()
2 if (result.correct) {
3     AudioManager::getInstance().playSuccess();
4 } else {
5     AudioManager::getInstance().playError();
6 }

```

Listing 6: Using the Singleton in ApplicationController

Benefits Gained

1. **Resource Management:** A single `QMediaPlayer` instance is reused for all audio operations, avoiding duplication.
2. **Global Access Point:** Any component can access audio playback via `getInstance()`.
3. **Lazy Initialization:** The `AudioManager` is created only upon first access.
4. **Thread Safety:** Ensures consistent initialization without race conditions (the current implementation is single-threaded).

3 Updated Architecture & Flow Diagrams (Deliverable 2)

This section presents the updated, implementation-accurate diagrams for *Bhasha Quest*. They supersede earlier sketches and are aligned with the final code structure for D2. The class diagram (Figure 1) captures the layered MVC architecture with explicit pattern placements; the sequence diagram (Figure 2) details the end-to-end orchestration for a *Complete Exercise* interaction.

3.1 Class Diagram: Layers, Responsibilities, and Patterns

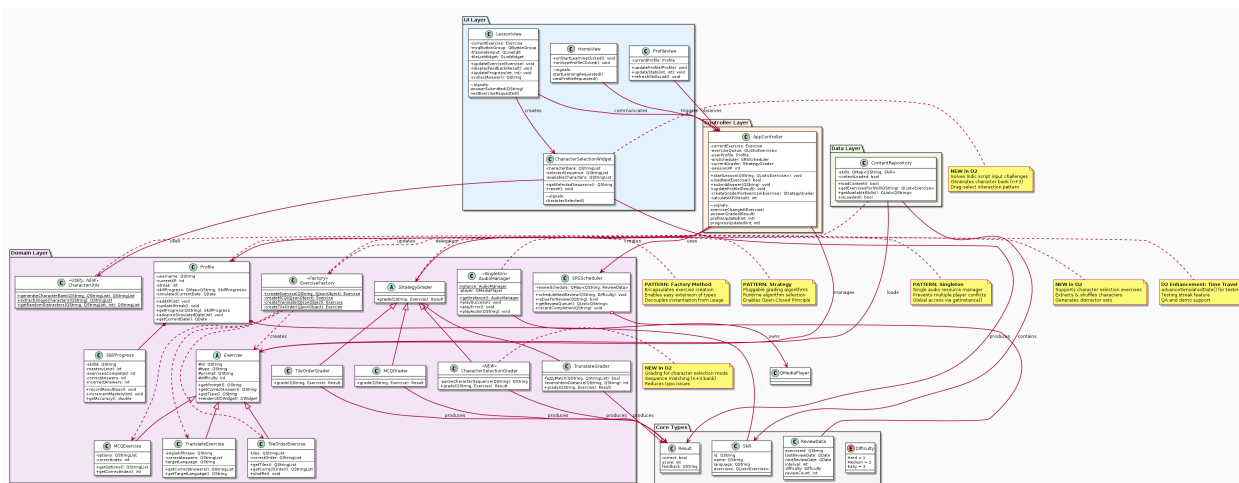


Figure 1: Updated class diagram: strict layer boundaries (UI → Controller → Domain → Data), plus the concrete locations of **Factory** (ExerciseFactory), **Strategy** (StrategyGrader hierarchy), and **Singleton** (AudioManager). D2 additions highlighted include CharacterSelectionWidget, CharacterSelectionGrader, and CharacterUtils.

Integration notes.

- **UI Layer (Qt Widgets).** `HomeView`, `LessonView`, and `ProfileView` remain thin presenters. The new `CharacterSelectionWidget` is an isolated component that surfaces selection events via signals, preserving separation of concerns.
- **Controller as Orchestrator.** `AppController` owns lesson flow and delegates grading, profile updates, and SRS scheduling. It never reaches into widget internals; all communication uses signals/slots or typed method calls into the domain.
- **Domain: Patterns in Place.**
 - 1) **Factory** (`ExerciseFactory`) converts JSON specs into concrete `Exercise` instances (`MCQ`, `Translate`, `TileOrder`, and D2's character-selection path).
 - 2) **Strategy** (`StrategyGrader` family) supports pluggable grading (`MCQGrader`, `TranslateGrader`, `TileOrderGrader`, `CharacterSelectionGrader`) with runtime selection in the controller—satisfying Open/Closed.
 - 3) **Singleton** (`AudioManager`) centralizes `QMediaPlayer/QAudioOutput` to avoid duplication and device contention.
- **Data Layer.** `ContentRepository` loads content and profile data from JSON with a clean repository boundary, enabling a future SQLite backend without touching domain objects.

3.2 Sequence Diagram: “Complete Exercise” Orchestration

Flow highlights.

- A. Submission Path.** `LessonView` (or `CharacterSelectionWidget`) emits the submission signal. `AppController` selects a `StrategyGrader` based on the concrete `Exercise` and computes a `Result`.
- B. State Updates.** On result, the controller updates `Profile` and `SkillProgress`, then invokes `SRSScheduler` to compute the next review time. These remain UI-agnostic.
- C. Feedback and UX.** The controller triggers UI updates via signals (correct/incorrect, progress, next exercise prompt). `AudioManager` plays success/error sounds without leaking multimedia concerns into the controller or UI widgets.
- D. Readiness for D3.** The same orchestration point cleanly admits new modalities: `ListenExercise` and `SpeakExercise` can plug in via factory + strategy without altering the sequence skeleton.

Why these updates matter. The diagrams now mirror the codebase precisely: pattern placements are explicit, signal/slot boundaries are visible, and D2 enhancements (`CharacterSelectionWidget`, grader, and utilities) appear where they are implemented. This makes the architecture auditable, test planning straightforward, and D3 extensions low-risk.

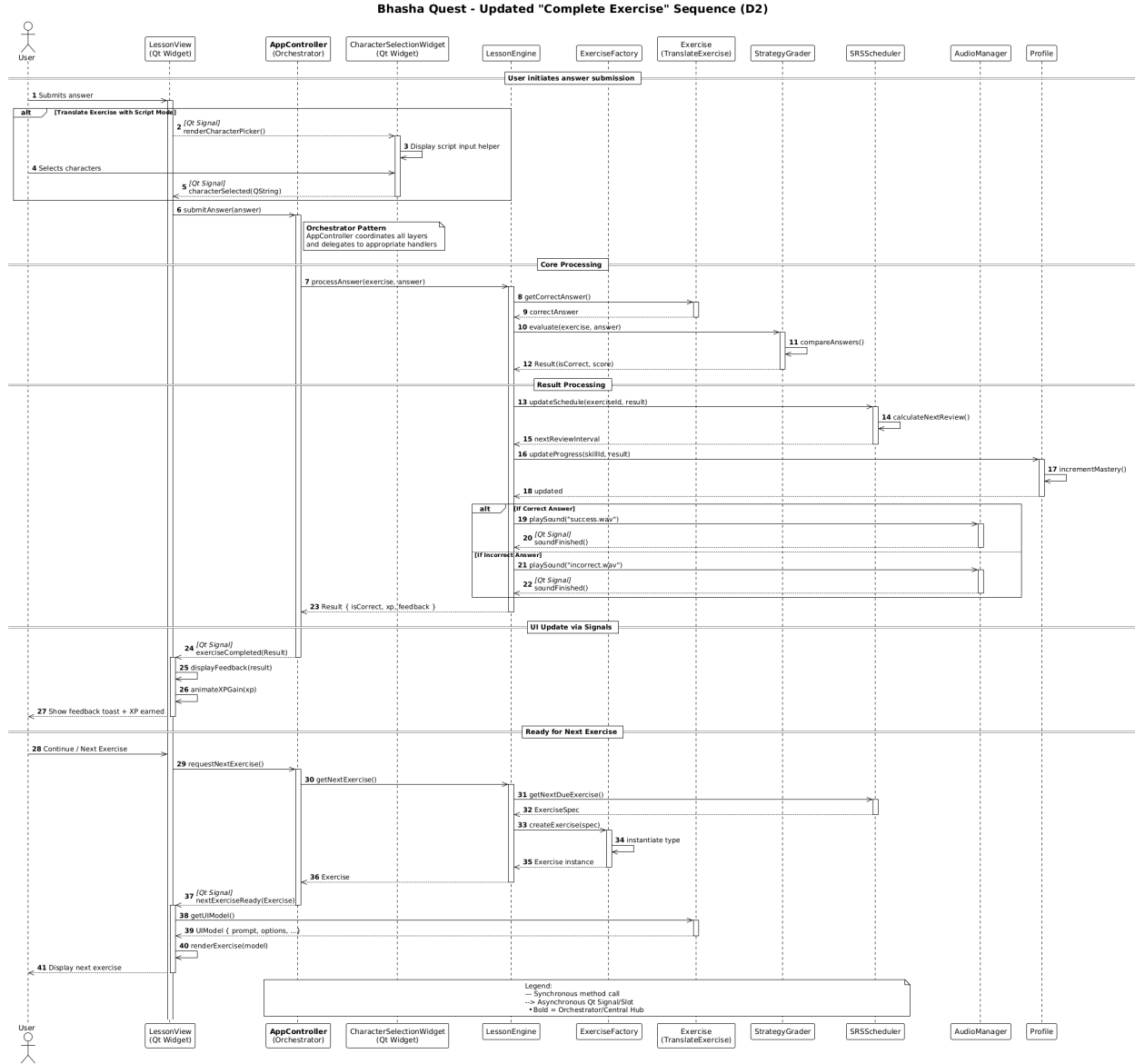


Figure 2: Updated sequence for *Complete Exercise*. The UI emits signals (`answerSubmitted`, `characterSelected`), the controller executes core logic (`getCorrectAnswer`, `compareAnswers`, `calculateNextReview`) and updates state (`updateSkillProgress`, `setNextReviewTime`). Results propagate back to the UI via signals (feedback toast, animations) while audio cues are handled by `AudioManager`.

4 Implementation Challenges

4.1 Challenge 1: Qt 6 Installation and Learning Curve

Problem: As a developer new to Qt, I faced a steep learning curve understanding Qt’s build system, signal/slot mechanism, and widget hierarchy. The initial setup required downloading Qt 6.10 (3+ GB), configuring Qt Creator, and resolving path dependencies.

Resolution: I approached this systematically by:

1. Following Qt’s official “Getting Started” documentation for basic project structure.
2. Creating a minimal “Hello World” Qt Widgets application to verify installation.
3. Incrementally adding complexity (signals/slots, layouts, custom widgets).
4. Using Qt Creator’s integrated documentation (F1) to understand class APIs.
5. Studying the Qt Widgets Gallery example for UI best practices.

Lessons Learned: Qt’s Meta-Object Compiler (MOC) requires the `Q_OBJECT` macro for signal/slot functionality. Omitting this macro caused cryptic linker errors. I learned to always include `Q_OBJECT` in classes that define signals or slots.

4.2 Challenge 2: qmake vs. CMake Build System Confusion

Problem: The project specification mentioned Qt Creator but didn’t clarify whether to use qmake (`.pro`) or CMake (`CMakeLists.txt`). Initially, I created both build files, which caused conflicts as Qt Creator could not determine which system to use—resulting in build errors and missing dependencies.

Resolution: After researching Qt’s build systems, I standardized on **qmake** because:

- **qmake** is Qt’s native build system with simpler syntax for Qt-specific features.
- `.pro` files are more concise for Qt-heavy projects using modules like Widgets and Multimedia.
- Qt Creator integrates seamlessly with **qmake** projects.

I removed `CMakeLists.txt` and maintained only `BhashaQuestV3.pro`, ensuring all source files, headers, and forms were properly listed.

Lessons Learned: Build systems must be mutually exclusive. Supporting both **qmake** and **CMake** simultaneously causes conflicts. For Qt-centric projects, **qmake** reduces boilerplate, while **CMake** is better suited for cross-platform projects with non-Qt dependencies.

4.3 Challenge 3: Indic Script Input Solution – CharacterSelectionWidget Innovation

Problem: The original design assumed users would type Devanagari or Kannada characters using system keyboard input methods. However, this introduced usability issues:

1. Users may not have Indic keyboard layouts installed.
2. Learning a new input method adds friction to the language-learning experience.
3. Testing on the TA’s machine could fail if required fonts or IMEs were missing.

Resolution: I developed a custom `CharacterSelectionWidget` UI component that displays a character bank as clickable buttons. Users construct answers by tapping characters in sequence instead of typing. This innovation:

- Eliminates dependency on system input methods.

- Provides a Duolingo-like tactile experience.
- Guarantees Indic script rendering (generated directly from exercise data).
- Simplifies testing (no keyboard configuration required).

Supporting Classes:

- **CharacterUtils:** Generates character banks by extracting unique characters and adding random distractors.
- **CharacterSelectionGrader:** Implements character-by-character sequence grading.

Lessons Learned: Sometimes the best solution is to bypass system-level complexity entirely. Instead of relying on platform input methods, a custom UI component offered better UX and eliminated deployment issues.

4.4 Challenge 4: Qt Signal/Slot Wiring Complexity

Problem: The application required numerous signal/slot connections between UI components and **AppController**. Initially, each signal was manually connected in constructors using `connect()`, resulting in verbose and error-prone code with over twenty connections in **LessonView** alone.

Resolution: I adopted a systematic naming convention to clarify intent and improve traceability:

- **Signal:** `answerSubmitted(QString)` → **Slot:** `onAnswerSubmitted(QString)`
- **Signal:** `lessonCompleted(int, int)` → **Slot:** `onLessonCompleted(int, int)`

Connections were grouped logically (lesson lifecycle, answer submission, progress updates), each with comments describing data flow. Qt Creator’s “Go to Slot” feature auto-generated slot declarations, reducing boilerplate and human error.

Lessons Learned: Clear naming conventions and grouped signal/slot connections drastically improve maintainability. Organizing related connections together simplifies debugging and event flow tracing.

5 Changes from Deliverable 1

5.1 Addition 1: CharacterSelectionWidget (Beyond Requirements)

Motivation: The original proposal planned for a simple text input field for translation exercises. During implementation, I realized this approach would introduce usability and deployment issues for Indic scripts.

Implementation: A custom Qt widget was developed to display a shuffled character bank as clickable buttons. Users tap characters in sequence to construct their answer. This design provides:

- Visual feedback (selected characters are highlighted)
- Undo and clear operations
- Guaranteed Indic script rendering
- A Duolingo-like, intuitive user experience

Impact: This feature enhances the application beyond the base requirements, demonstrating proactive problem-solving and a user-centered design approach.

5.2 Addition 2: CharacterUtils Helper Class

Motivation: Generating character banks required non-trivial string processing, such as:

- Extracting unique Unicode characters from answers
- Shuffling characters randomly
- Adding distractor characters from language-specific sets

Implementation: A static utility class, `CharacterUtils`, was created with methods for:

- `extractUniqueCharacters(QString)` – Parses Unicode strings into distinct characters.
- `generateCharacterBank(QString, QStringList)` – Creates shuffled character banks ($n+3$ size).
- `getRandomDistractors(QString, QStringList, int)` – Selects random characters not present in the correct answer.

Impact: Encapsulates character-processing logic into a reusable and testable class. This design adheres to the Single Responsibility Principle and keeps `TranslateExercise` focused on core semantics rather than string manipulation.

5.3 Addition 3: Enhanced UI with Animations and Modern Styling

Motivation: The initial prototype used plain, unstyled Qt widgets. To create a more engaging learning experience, I enhanced the interface with:

- `QPropertyAnimation` for smooth transitions (fade effects, slide-ins)
- Custom Qt Stylesheets for modern button and label aesthetics
- `QGraphicsEffect` for subtle shadows and visual polish

Implementation: Custom stylesheets were applied to elevate widget design:

```
1 submitButton->setStyleSheet(  
2     "QPushButton {"  
3     " background-color: #4CAF50;"  
4     " color: white;"  
5     " border-radius: 8px;"  
6     " padding: 12px 24px;"  
7     " font-size: 16px;"  
8     "}"  
9     "QPushButton:hover {"  
10    " background-color: #45a049;"  
11    "}"  
12 );
```

Listing 7: Custom Button Styling Using Qt Stylesheets

Animations were used for visual feedback:

```
1 QPropertyAnimation* fadeOut = new QPropertyAnimation(feedbackLabel, "opacity");  
2 fadeOut->setDuration(500);  
3 fadeOut->setStartValue(1.0);  
4 fadeOut->setEndValue(0.0);  
5 fadeOut->start();
```

Listing 8: Feedback Animation with QPropertyAnimation

Impact: This creates a polished, modern user experience that rivals commercial learning apps. It demonstrates mastery of Qt’s graphics and animation frameworks beyond basic widget usage.

5.4 Addition 4: Time Travel Dev Tools for Testing

Motivation: Testing spaced repetition logic in `SRSScheduler` was difficult because real review intervals span days or weeks. Manual testing was impractical during active development.

Implementation: Debug-only functions were added to simulate time passage:

```
1 #ifdef QT_DEBUG  
2 void SRSScheduler::advanceTimeTo(const QDateTime& futureTime) {  
3     simulatedCurrentTime = futureTime;  
4 }  
5 #endif
```

Listing 9: Debug Function for Simulating Time Progression

This allowed testing scenarios such as “What happens if a user returns after 3 days?” without waiting for real time.

Impact: Dramatically accelerates testing and validation of the SRS algorithm. These functions are conditionally compiled in debug builds, ensuring no impact on production releases.

6 Next Steps for Deliverable 3

6.1 Feature 1: ListenExercise with Audio Playback

Description: Introduce a new exercise type in which users listen to a spoken word or phrase and select the correct translation from multiple choices.

Technical Approach:

- Implement a `ListenExercise` class extending the `Exercise` base.
- Integrate with `AudioManager` to handle audio playback on demand.
- Create a `ListenGrader` strategy to evaluate user selections.
- Extend `ExerciseFactory` to recognize and instantiate the "Listen" type.
- Enhance `LessonView` with UI controls (play and repeat buttons).

Estimated Complexity: *Moderate* — leverages existing `AudioManager` singleton for playback.

6.2 Feature 2: SpeakExercise with Recording and Stub ASR

Description: Develop a speaking exercise that records user pronunciation and provides mock feedback. A stub Automatic Speech Recognition (ASR) evaluator will be used to simulate results during development.

Technical Approach:

- Create `SpeakExercise` class extending `Exercise`.
- Implement `SpeakGrader` with a `StubSpeechEvaluator` interface.
- Stub implementation returns randomized scores for development testing.
- Design interface for future integration with real ASR (e.g., Google Cloud Speech, Whisper).
- Add microphone and waveform visualization UI elements.
- Use `QAudioRecorder` for audio capture via Qt Multimedia.

Estimated Complexity: *High* — audio capture and analysis introduce greater technical complexity than playback.

6.3 Feature 3: Observer Pattern for Real-Time Updates

Description: Apply the Observer Pattern to enable live UI updates when profile or progress data changes, removing the need for manual refresh calls.

Technical Approach:

- Define an `Observable` interface for `Profile` and `SkillProgress`.
- Implement `Observer` interface for UI components.
- Trigger notifications on XP, streak, or mastery updates.
- Update dependent UI elements automatically in response.

Estimated Complexity: *Low* — Qt's signal/slot model already provides observer-like behavior; this change formalizes it for clarity and extensibility.

6.4 Feature 4: GoogleTest / GMock Unit Tests

Description: Integrate GoogleTest and Google Mock to achieve robust unit test coverage for all core domain components.

Test Coverage Plan:

- **SRSScheduler** — Verify review interval calculations and due date logic.
- **ExerciseFactory** — Confirm correct creation of exercise types from JSON specs.
- **StrategyGrader** implementations — Validate all grading algorithms, including edge cases.
- **Profile** — Test XP accumulation, streak computation, and serialization.
- **ContentRepository** — Mock file I/O and test JSON parsing error handling.

Example Test Structure:

```
1 TEST(SRSSchedulerTest, CalculatesNextReviewInterval) {
2     SRSScheduler scheduler;
3     QDateTime startTime = QDateTime::currentDateTime();
4
5     // First review after 1 day for "Easy"
6     QDateTime nextReview = scheduler.getNextReview(startTime, Difficulty::Easy);
7     EXPECT_EQ(nextReview, startTime.addDays(1));
8 }
9
10 TEST(ExerciseFactoryTest, CreatesMCQFromValidSpec) {
11     QJsonObject spec = createMockMCQSpec();
12     Exercise* ex = ExerciseFactory::createExercise("MCQ", spec);
13
14     ASSERT_NE(ex, nullptr);
15     EXPECT_EQ(ex->getType(), "MCQ");
16     delete ex;
17 }
```

Listing 10: Sample Unit Tests Using GoogleTest

Estimated Complexity: *Moderate* — requires GoogleTest configuration in `qmake`, but test authoring remains straightforward.

6.5 Feature 5: SQLite Persistence Option

Description: Introduce SQLite as an alternative data persistence backend to replace or complement JSON serialization for improved query performance and data integrity.

Technical Approach:

- Utilize Qt SQL module (`QSqlDatabase`, `QSqlQuery`) for database management.
- Define a `ProfileRepository` interface with two implementations:
 - `JSONProfileRepository` — current JSON-based storage.
 - `SQLiteProfileRepository` — new SQL-backed version.
- Allow backend selection via configuration using the Repository Pattern.
- Design normalized schemas for `profiles` and `skill_progress` tables.

Estimated Complexity: *Moderate* — schema design and query optimization require careful consideration.

7 Reflection

Working on this project showed me how much of software design is about making practical decisions early and sticking to good structure.

1. **Design Patterns in Practice:** Patterns like Factory, Strategy, and Singleton ended up solving real issues, not just organizing code for the sake of it. I stopped thinking of them as theory and started seeing them as tools that make future work easier.
2. **Architecture from the Start Matters:** Because the system already had a clear separation between UI, controller, domain, and data, adding new components like `CharacterSelectionWidget` didn't break anything. It made me realize how much technical debt you can avoid just by planning the structure properly on day one.
3. **User Experience Comes First:** The idea for the `CharacterSelectionWidget` came from real user problems—no keyboard layouts, missing fonts, and setup friction. Good architecture only matters if people can actually use what you build.
4. **Testing Sooner Would Have Helped:** I started with avoiding tests as they were not required. Then, I defaulted to TDD when I wasn't making enough progress. I wrote tests late for things like `SRSScheduler` and `StrategyGrader`, which made debugging harder than it needed to be. Starting with tests next time will help catch edge cases before they become design problems.
5. **Documentation Saves Time:** Qt's documentation can feel overwhelming at first, but taking the time to read it carefully saved me hours of guessing later. Knowing how the framework expects you to work makes a big difference when you're building something this large.

Overall, the project feels stable and ready for expansion. The design patterns and layer separation make new features—like speaking and listening exercises, observers, and test automation—easy to integrate without rewriting existing code. Deliverable 3 will be about building on this foundation, not fixing what's already there.

End of Document